

Design Entscheidungen

Verwendung von NX

Das Projekt verwendet das *NX*-Entwicklungs-Framework für die Verwaltung des *Monorepo*, der Codegenerierung und der Build- und Test-Workflows.

Begründung: Die Verwendung von *NX* bietet mehrere Vorteile für das Projekt, einschließlich:

- *Monorepo*-Verwaltung: *NX* erleichtert die Verwaltung von Code in einem *Monorepo*, da es eine konsistente Struktur von Code bietet und Abhängigkeiten zwischen Projekten innerhalb des *Monorepo* verwaltet.
- Codegenerierung: *NX* bietet Tools für die automatisierte Generierung von Code, einschließlich der Generierung von *Scaffold* für neue *Komponenten*, *Services* und *Libraries*.
- Build- und Test-Workflows: *NX* vereinfacht die Konfiguration von Build- und Test-Workflows für das gesamte *Monorepo*, einschließlich der Integration von *Cypress* für Tests für die Verwendung in einer Continuous-Integration/Continuous-Delivery-Pipeline (*CI/CD*).
- Skalierbarkeit: *NX* ermöglicht es eine *monolithische Architektur* für das Projekt zu verwenden, indem es eine klare Struktur für Code und Abhängigkeiten bietet. Im Rahmen unseres Projekts beinhaltet das *Monorepo* sowohl das *Frontend* als auch das *Backend* der *Anwendung*, sowie *E2E-Tests*.

Verwendung von Client Server

Die *Client-Server-Architektur* wird für die Implementierung der *Anwendung* verwendet.

Begründung: Die Verwendung der *Client-Server-Architektur* bietet mehrere Vorteile für das Projekt, einschließlich:

- Skalierbarkeit: Durch die Trennung der *Anwendung* in *Frontend*- und *Backend-Komponenten* kann jeder Teil der *Anwendung* unabhängig skaliert werden, um eine höhere Last zu bewältigen, ohne die Leistung des anderen Teils zu beeinträchtigen.
- Sicherheit: Die Verwendung einer *Client-Server-Architektur* ermöglicht es, Sicherheitsmaßnahmen wie Zugriffskontrolle und Verschlüsselung an der *Backend*-Seite zu implementieren, um die *Anwendung* vor Angriffen zu schützen. Wir können somit beispielsweise dem `_Veranstalter_` eines Events mehr Rechte geben als einem Teilnehmer.
- Wiederverwendbarkeit: Die Trennung der *Anwendung* in *Frontend*- und *Backend-Komponenten* ermöglicht es, die Wiederverwendung von *Komponenten* besser zu organisieren. In unserem Projekt werden *Frontend-Komponenten* nur im *Frontend* wiederverwendet und *Backend-Komponenten* nur im *Backend* wiederverwendet.

Verwendung eines RESTful-Webservices

Die *Anwendung* verwendet *RESTful-Webservices* zur Kommunikation zwischen dem *Frontend* und dem *Backend*.

Begründung: Die Verwendung eines *RESTful-Webservices* bietet mehrere Vorteile für das Projekt, einschließlich:

- Interoperabilität: *RESTful-Webservices* verwenden standardisierte HTTP-Methoden und Datenformate, was es verschiedenen Systemen und Programmiersprachen ermöglicht, miteinander zu kommunizieren. In unserem Projekt muss ein *Angular Frontend* (geschrieben *TypeScript*) mit einem *Java Backend* kommunizieren.
- Skalierbarkeit: *RESTful-Webservices* ermöglichen die Skalierung der *Anwendung*, indem sie das *Frontend* und das *Backend* entkoppeln und es dem *Backend* ermöglichen, mehrere Anfragen gleichzeitig zu verarbeiten. Unabhängig davon wie viele Clients das *Frontend* der *Party Time Anwendung* verwenden, kann das *Backend* die Anfragen verarbeiten.
- Einfachheit: *RESTful-Webservices* sind einfach zu implementieren und zu nutzen, da sie auf standardisierten Methoden und Datenformaten basieren.

Verwendung von Java

Die *Anwendung* verwendet *Java* als Programmiersprache für die Implementierung des *Backend*.

Begründung:

Die Verwendung von *Java* bietet mehrere Vorteile für das Projekt, einschließlich:

- Interoperabilität: *Java* ist eine Programmiersprache, die es ermöglicht, mit dem *Spring Boot-Framework* für die Implementierung des *Backend-Servers* zu arbeiten.
- Wiederverwendbarkeit: *Java* ist eine objektorientierte Programmiersprache, die die Wiederverwendung von Code und *Komponenten* fördert. Authentifizierung ist beispielsweise ein Baustein vieler unserer Anforderungen, die nur einmal zentral im *Java-Backend* implementiert werden muss.

Verwendung von Spring Boot

Die *Anwendung* verwendet das *Spring Boot-Framework* als Basis für die Implementierung des *Backend-Servers*.

Begründung: Die Verwendung von *Spring Boot* bietet mehrere Vorteile für das Projekt, einschließlich:

- Schnelle Entwicklung: *Spring Boot* bietet vorgefertigte Module und Konfigurationen, welche die Implementierung der *Anwendung* beschleunigen kann.

- Einfache Konfiguration: *Spring Boot* verwendet Konventionen und automatisierte Konfigurationen für die einfache Konfiguration verschiedener Aspekte der *Anwendung*, ohne dass manuelle Eingriffe erforderlich sind.
- Erweiterbarkeit: *Spring Boot* ist einfach zu erweitern und bietet eine Vielzahl von Erweiterungen und Plugins, welche die Implementierung von zusätzlichen Funktionen erleichtern.

Verwendung von Maven

Das Projekt verwendet Apache *Maven* als Build-Management-Tool des *Java-Backend-Servers*, um den Build-Prozess zu automatisieren, die Abhängigkeiten zu verwalten und die Bereitstellung des *Backend-Servers* zu erleichtern.

Begründung: Die Verwendung von *Maven* bietet mehrere Vorteile für das Projekt, einschließlich:

- Automatisierung: *Maven* automatisiert den Build-Prozess und ermöglicht eine schnelle, zuverlässige und wiederholbare Erstellung der *Anwendung*.
- Abhängigkeitsverwaltung: *Maven* verwaltet die Abhängigkeiten der *Backend-Anwendung* automatisch und stellt sicher, dass die richtigen Versionen von Abhängigkeiten verwendet werden.
- Erweiterbarkeit: *Maven* bietet eine Vielzahl von Plugins, die es einfach machen, zusätzliche Funktionalitäten zu integrieren, z.B. zur Code-Qualitätsprüfung, Testausführung oder Dokumentation.

Verwendung von H2

Das Projekt verwendet eine integrierte *H2*-Datenbank für die lokale Entwicklung und Tests. *H2* ist eine leichte, schnelle und plattformunabhängige *Datenbank*, welche in *Java* geschrieben wurde und eine einfache Vergleichbarkeit mit anderen JDBC-Datenbanken ermöglicht.

Begründung:

Die Verwendung einer *H2* Datenbank bietet mehrere Vorteile für das Projekt, einschließlich:

- Einfache Einrichtung: *H2* kann als eingebettete *Datenbank* direkt im Projekt verwendet werden, was eine schnelle Einrichtung und Konfiguration ermöglicht.
- Schnelligkeit: *H2* ist eine schnelle *Datenbank*, die keine langen Initialisierungszeiten benötigt und schnell auf Anfragen reagiert.
- Plattformunabhängigkeit: *H2* ist in *Java* geschrieben und somit plattformunabhängig. Es kann auf verschiedenen Betriebssystemen eingesetzt werden.
- Kompatibilität: *H2* ist vergleichbar mit anderen JDBC-Datenbanken und ermöglicht so eine einfache Migration zu anderen, evtl. für den Betrieb besser geeigneten Datenbanken.

- Testfähigkeit: *H2* eignet sich besonders gut für Integrationstests und automatisierte Tests, da die eingebettete Datenbank leicht in Testumgebungen gestartet werden kann.

Verwendung von Single-Page-Anwendung

Das Projekt implementiert eine Single-Page-Anwendungen (*SPA*)-Architektur für das Frontend.

Begründung: Die Verwendung von einer *SPA* bietet mehrere Vorteile für das Projekt, einschließlich:

- Trennung des *Frontend* vom *Backend*, was die Wartbarkeit erhöht und die Entwicklung erleichtert. Die *SPA* kann unabhängig vom *Backend* entwickelt werden, solange die Schnittstellen zwischen *Frontend* und *Backend* definiert sind. In diesem Projekt werden die Schnittstellen mit *Swagger* definiert.
- Verbesserte Benutzererfahrung, da das Laden und Navigieren zwischen den Seiten nicht zu vollständigen Neuladungen führt, ist die *Anwendung* reaktionsschneller und bietet eine flüssige Benutzererfahrung.

Verwendung von Angular

Das Projekt verwendet das *Angular-Framework* für die Implementierung der *Frontend-Anwendung*.

Begründung: Die Verwendung von *Angular* bietet mehrere Vorteile für das Projekt, einschließlich:

- Struktur: *Angular* bietet eine klare Struktur für die Organisation von Code.
- Templates: *Angular* bietet eine Vorlagen-Engine für die Erstellung von dynamischen und reaktiven Benutzeroberflächen.
- Komponenten: *Angular* arbeitet mit *Komponenten*, die unabhängig voneinander entwickelt werden können und wiederverwendbar sind (z.B. Event-Liste, Event-Detailansicht, Event-Erstellung)
- Ökosystem: *Angular* verfügt über ein Ökosystem von Bibliotheken und Tools, welche die Entwicklung und Wartung von unserer *Anwendung* erleichtert

Verwendung von NPM (Node Package Manager)

Das Projekt verwendet den Node Package Manager (*NPM*) als zentrales Tool zur Verwaltung von Abhängigkeiten und zur Automatisierung von Abläufen im *Frontend-Entwicklungsprozess*.

Begründung: Die Verwendung von *NPM* bietet mehrere Vorteile für das Projekt, einschließlich:

- Verwaltung von Abhängigkeiten: *NPM* erleichtert die Verwaltung von Abhängigkeiten im Projekt und sorgt dafür, dass alle für das *Frontend* benötigten Bibliotheken und Tools auf dem neuesten Stand sind und gleiche Versionen konsistent verwendet werden (z. B. gleiche Version von *Angular* und *NGRX*).
- Automatisierung von Prozessen: *NPM* ermöglicht es Entwicklern, wiederholende Aufgaben im Entwicklungsprozess zu automatisieren, wie z.B. die Ausführung von Tests oder die Bereitstellung von Builds.
- Ökosystem: *NPM* verfügt über ein Ökosystem von Bibliotheken und Tools, die die Entwicklung und Wartung von Anwendungen erleichtern.
- Skalierbarkeit: *NPM* ist skalierbar und kann mit der Größe und Komplexität des Projekts wachsen, wodurch eine einfache Verwaltung und Entwicklung des Projekts möglich wird.

Verwendung von NGRX

Das Projekt verwendet das *NGRX*-Framework zur Implementierung eines Stores und zur Verwaltung des Anwendungsstatus in der *Angular-Komponenten*.

Begründung: Die Verwendung von *NGRX* bietet mehrere Vorteile für das Projekt, einschließlich:

- Vorhersehbarkeit: *NGRX* basiert auf dem Redux-Muster, das eine klare Trennung von Anwendungsstatus und Benutzerinteraktion ermöglicht. Dadurch wird die Vorhersehbarkeit des Verhaltens der *Anwendung* verbessert und das Erstellen von Tests erleichtert.
- Architektur: *NGRX* bietet eine klare Architektur für die Implementierung der *Frontend-Anwendung*, welche die Wartbarkeit erhöht und die Entwicklung erleichtert (z. B. automatisches Starten von *API* Anfragen und Verarbeiten von *API* Antworten).

Verwendung von Cypress

Das Projekt verwendet das *Cypress*-Testing-Framework für die Automatisierung von *E2E-Tests* im *Frontend*.

Begründung: Die Verwendung von *Cypress* bietet mehrere Vorteile für das Projekt, einschließlich:

- Integration: *Cypress* lässt sich nahtlos in die Continuous-Integration/Continuous-Delivery-Pipeline des Projekts integrieren und ermöglicht es dem Team, automatisierte Tests zu erstellen und auszuführen, bevor Code übernommen wird.
- Dokumentation: Im Verlauf von *Cypress* Tests kann das Framework Screenshots und Videos erstellen, die der Dokumentation der Tests dienen. Außerdem wird eine bessere Nachvollziehbarkeit des Anwendungsverhaltens im Bezug auf die in UML Aktivitätsdiagramm beschriebenen Abläufen ermöglicht.

Verwendung von Tailwind CSS

Das Projekt verwendet das *Tailwind CSS* Framework für die Gestaltung des Frontends. *Tailwind CSS* ist ein Utility-First CSS-Framework, das es Entwicklern ermöglicht, responsive Benutzeroberflächen zu erstellen.

Begründung: Die Verwendung von *Tailwind CSS* bietet mehrere Vorteile für das Projekt, einschließlich:

- Effizienz: *Tailwind CSS* bietet eine Vielzahl von vordefinierten Utility-Klassen.
- Flexibilität: *Tailwind CSS* ist sehr flexibel und erlaubt es Designs zu erstellen, ohne aufwendige CSS-Regeln schreiben zu müssen.
- Responsivität: *Tailwind CSS* verfolgt den Mobile-First Ansatz und erleichtert die Erstellung von responsiven Designs für verschiedene Geräte.
- Design: Einheitliches Design für alle *Frontend*-Komponenten der Party Time Plattform.

Verwendung von GitHub

Das Projekt verwendet *GitHub* als Code-Repository und Kollaborationsplattform für *Issue*-Tracking.

Begründung: Die Verwendung von *GitHub* bietet mehrere Vorteile für das Projekt, einschließlich:

- Kollaboration: *GitHub* bietet eine benutzerfreundliche und intuitive Plattform für die Zusammenarbeit zwischen Entwicklern, die es einfach macht, Code zu teilen, zu kommentieren und zusammenzuarbeiten.
- Versionskontrolle: *GitHub* bietet eine leistungsfähige Versionskontrollfunktion, die es ermöglicht, verschiedene Versionen des Codes zu speichern und bei Bedarf wiederherzustellen.
- Code-Review: *GitHub* bietet eine integrierte Code-Review-Funktion, die es Entwicklern ermöglicht, den Code zu überprüfen und Feedback zu geben.
- Integrationen: *GitHub* bietet Integrationen mit einer Vielzahl von Tools und Services, welche die Entwicklung und Wartung der Party Time Plattform erleichtern, wie z.B. Continuous Integration/Continuous Deployment (*CI/CD*) Tools und *NX*.

Verwendung von PlantUML

Das Projekt verwendet *PlantUML* als Werkzeug für die Erstellung von UML-Diagrammen.

Begründung: Die Verwendung von *PlantUML* bietet mehrere Vorteile für das Projekt, einschließlich:

- Einfache Syntax: *PlantUML* verwendet eine einfache, textbasierte Syntax, um UML-Diagramme zu erstellen.

- **Flexibilität:** *PlantUML* unterstützt verschiedene Arten von UML-Diagrammen, einschließlich Klassendiagrammen, Sequenzdiagrammen und Zustandsdiagrammen, sowie benutzerdefinierte Diagrammtypen. In diesem Projekt wird mit *PlantUML* das Datenbankschema der *Anwendung* modelliert, sowie mit Sequenz- und Ablaufdiagrammen der Ablauf von Programm-Abschnitten beschrieben.
- **Integration:** *PlantUML* kann leicht in andere Tools und Workflows integriert werden, z.B. in IDEs, CI/CD-Pipelines oder Dokumentationsprozesse.
- **Export:** *PlantUML* bietet eine Vielzahl von Exportoptionen, um die erstellten Diagramme in verschiedenen Formaten zu exportieren, z.B. als Bild-, PDF- oder ASCII-Art-Datei. In diesem Projekt werden die Diagramme als PNG-Bilder exportiert, welche in der Bibliotheken-Dokumentation referenziert werden.

Verwendung von Google Maps

Das Projekt verwendet die *Google Maps API* zur Integration von interaktiven Karten und Geodaten in die *Anwendung*. Die *Google Maps API* ist eine verbreitete *API*, die es ermöglicht, Karten, Standorte, Wegbeschreibungen und geografische Daten in die *Anwendung* zu integrieren.

Begründung: Die Verwendung von *Google Maps* bietet mehrere Vorteile für das Projekt, einschließlich:

- **Umfangreiche Funktionen:** Die *Google Maps API* ermöglicht die Darstellung von Veranstaltungsorten auf einer Karte, die Berechnung von Wegbeschreibungen und die Anzeige von Standorten in Satelliten- und Straßenansicht.
- Die *Google Maps API* bietet Zugriff auf aktuelle Karten- und Geodaten um genaue und aktuelle Informationen wie das aktuelle Verkehrsaufkommen oder Fahrradwege bereitzustellen

Erforderliche persönliche Daten von Benutzern

Für die Authentifizierung von Benutzern sowie für soziale Komponenten der Party Time Plattform muss die Plattform *persönliche Daten* verarbeiten. Pro Nutzer wird mindestens ein Anzeigenname, eine E-Mail Adresse sowie ein Passwort verarbeitet.

Begründung:

- **Anzeigenname:** Ein frei vom Nutzer gewählter Anzeigenname, der anderen Nutzern beispielsweise in Party-Teilnehmerlisten gezeigt wird. Dieser ist frei wählbar und nicht etwa mit einem Echter-Name-Zwang verbunden, da letztendlich Nutzer selbst dafür verantwortlich sind, dass andere Party-Gäste einem Anzeigenamen einer Person zuordnen können. Wird

beispielsweise ein Nickname als Anzeigename verwendet, stellt dies für andere Partygäste grundsätzlich kein Problem dar.

- E-Mail Adresse: Da Anzeigenamen frei wählbar sind und daher eventuell auch mehrere Nutzer den gleichen Anzeigenamen verwenden können, benötigt der Anmelde-Vorgang eine E-Mail Adresse von Nutzern. Bei einzigartigen E-Mail Adressen besteht Sicherheit, dass mehrere Nutzer nicht dieselbe E-Mail Adresse verwenden.
- Passwort: Um davor zu schützen, dass sich dritte Personen mit E-Mail Adressen anderer Nutzer anmelden, erfolgt eine personalisierte Anmeldung an der Party Time Plattform ausschließlich durch die Eingabe von E-Mail Adresse und vom Nutzer gewählten Passwort.

Anzeigename-Richtlinien

Für Anzeigenamen gelten die folgenden Richtlinien:

- Ein Anzeigename hat eine Länge zwischen 5 und 20 Zeichen
- Alle Zeichen sind zulässig

Begründung:

5 - 20 Zeichen bieten Nutzern eine ausreichende Länge ihres Anzeigenamen, um einen ihren Wünschen entsprechenden Anzeigename zu wählen.

Für einen "Echte-Welt" Einsatz würden wir Zeichen wie ASCII- und UTF-Kontrollsequenzen, führende und folgende Leerzeichen, sowie mehrfach aufeinander folgende Leerzeichen verbieten. Die Entwicklung eines Algorithmus für die Prüfung dieser Regeln (insbesondere der Ausschluss aller nicht erlaubten Zeichen) würde allerdings den Umfang der *Anwendung* in seiner ersten Ausbaustufe sprengen.

E-Mail-Adresse Richtlinien

E-Mail Adressen müssen dem HTML Standard für Valide E-Mail Adressen entsprechen und vom *Angular* Framework als valide empfunden werden.

Begründung:

Das verwendete *Angular* Framework hat einen email Validator für diesen Einsatzzweck, der zur Validierung von E-Mail-Adressen in Formularen mit geringem Entwickleraufwand eingesetzt werden kann.

Passwort-Richtlinien für Benutzer

Passwörter für Benutzer müssen die folgenden Anforderungen erfüllen:

- Mindestens 8 Zeichen lang sein
- Maximal 30 Zeichen lang sein
- 4 Zeichenarten verwenden (je min. 1 Großbuchstaben, Kleinbuchstaben, Ziffer und Sonderzeichen)

- Die 1 benötigte Ziffer und das 1 benötigte Sonderzeichen dürfen nicht am Anfang oder Ende des Passworts stehen. Auch eine Folge von Sonderzeichen und Ziffern am Anfang und Ende erfüllen die Regel nicht.
- Als Sonderzeichen anerkannt werden: `!@#$%^&*()=?{}[]` und Leerzeichen

Begründung:

Einhaltung der Bundesamt für Sicherheit in der Informationstechnik (BSI): Sichere Passwörter erstellen Empfehlung

Löschen eines Accounts

Beim Löschen eines Accounts werden alle damit verbundenen Daten gelöscht, einschließlich der Absage und Löschung von Events, bei denen der Nutzer als Organisator fungiert.

Begründung:

Schutz der Privatsphäre, der Datensicherheit und einer konsistenten Benutzererfahrung.

Event-Richtlinien

Für die Veranstaltung eines Events muss ein verifizierter Nutzer die folgenden Informationen bereitstellen:

- Einen 5 - 50 Zeichen langen Event-Namen
- Eine 4 - 25 Zeichen lange Adresszeile für Straße und Hausnummer. Diese kann Buchstaben, Leerzeichen, Punkte, Bindestriche und Zahlen beinhalten.
- Eine 5-stellige Postleitzahl. Diese darf nur Ziffern beinhalten, wird aber letztendlich als String interpretiert um führende Nullen nicht zu verlieren.
- Einen 3 - 20 stelligen Ort. Darf Groß- und Kleinbuchstaben, Leerzeichen, Punkte und Bindestriche enthalten.
- Ein 3 - 20 stelliges Land. Darf Groß- und Kleinbuchstaben sowie Leerzeichen und Bindestriche enthalten.
- Eine Uhrzeit im 24h Format. Stunden von 00 - 23, Minuten von 00 - 59.
- Ein Datum im Format zwei Ziffern für den Tag, zwei Ziffern für den Monat, 4 Ziffern fürs Jahr

Die Plattform bietet außerdem eine optionale, bis zu 25 Zeichen lange Adresszusatz Zeile unter der Zeile für Straße und Hausnummer, für die eingabe von zusätzlichen Infos wie z. B. ein Stockwerk oder eine Wohnungs-Nummer.

Begründung:

- 5 - 50 Zeichen für den Event-Namen werden als angemessen angesehen. Dies sind weder zu wenig Zeichen (was zur Verwirrung der eingeladenen Gäste führen könnte), noch sind es zu viele Zeichen (es soll nur ein Event-Name, keine Event-Beschreibung eingegeben werden).

- 4 - 25 Zeichen für die Straße und Hausnummer Adresszeile werden als ausreichend angesehen.
- 5-stellige Postleitzahl restriktiert den Einsatz der Plattform auf Länder, in denen Postleitzahlen 5 Zeichen lang sind. Dies wird für die erste Version der Plattform als akzeptabel angesehen und kann in weiteren Ausbaustufen erweitert werden.
- 3 - 20 stelliger Ort und Land mit Groß- und Kleinbuchstaben sowie Bindestrichen sind akzeptabel für die erste Version der Plattform. Es ist bekannt, dass die Plattform für den Einsatz in fremden Ländern mehr Zeichen-Typen unterstützen sollte. Dies kann in einer späteren Ausbaustufe erweitert werden.
- Für die Uhrzeit und das Datum wird die *Angular* Standard-Funktionalität namens *DatePipe* verwendet. Das Datums-Format richtet sich zunächst nach dem deutschen Zeit-Standard, der in DIN 5008 (z.B. TT.MM.YYYY) definiert ist. In einer späteren Ausbaustufe können im Rahmen einer spezialisierten Lokalisierung noch weitere Zeit- und Datums-Formate integriert werden.
- Die optionale, bis zu 25 Zeichen lange Adresszusatzzeile wird als notwendig angesehen, um insbesondere in Ballungsgebieten den Teilnehmern eines Events das auffinden des Veranstaltungsortes zu vereinfachen.

Unterscheidung zwischen Veranstalter und Teilnehmer eines Events

Ein Nutzer kann ein Event als *Veranstalter* veranstalten oder als *Teilnehmer eines Events* an einem Event teilnehmen. Die Plattform unterscheidet zwischen diesen beiden Rollen und bietet dem Nutzer unterschiedliche Ansichten und Funktionen an.

Begründung:

- Der *Veranstalter* kann das Event bearbeiten und löschen. Der *Teilnehmer eines Events* kann dies nicht.
- Der *Veranstalter* kann die Teilnehmerliste einsehen und Teilnehmer zu dem Event ein- und ausladen. Der *Teilnehmer eines Events* kann dies nicht.
- Der *Teilnehmer eines Events* kann dem Event zu- oder absagen. Der *Veranstalter* kann dies nicht.
- Der *Teilnehmer eines Events* kann die Adresse des Events als integrierte *Google Maps* Karte einsehen. Der *Veranstalter* kann dies nicht, da vorausgesetzt wird, er kennt die Adresse bereits.